RCOUNT: Reproducible Election Count Packages

Gio Della-Libera

Draft — June 28, 2026

Abstract

Election results become public facts through a mixture of voting-system exports, batch accounting, canvass decisions, recounts, public records, and legal certification. This paper introduces RCOUNT, a reproducible package format and verifier contract for replaying election-count claims from public evidence. RCOUNT does not certify elections and does not prove voter choices. Instead, it binds normalized contest summaries, reporting units, batches, lineage events, source hashes, and privacy-safe proof sketches into a package that third parties can verify mechanically. We describe the package layers, the core reconciliation checks, the optional RPLAN boundary for district aggregation, and a synthetic fixture suite that includes both positive and negative multi-election cases.

1 Introduction

Election-count publication is not a single computation. Election-night returns are often unofficial; late mail ballots, provisional adjudication, ballot cures, write-in review, duplication, recounts, and court orders may legitimately change totals before certification. Public trust therefore depends on two ideas that must be held together: the public should be able to replay arithmetic from available evidence, and the software doing the replay must not pretend to make the legal judgment that election officials and courts make.

RCOUNT is a proposed substrate for that replay. It packages election-count records into a stable directory layout, uses canonical hashes to bind normalized records to source evidence, and emits verification transcripts for mechanical checks. Its target is narrower than end-to-end cryptographic voting and broader than a single vendor export parser. RCOUNT asks: given the public artifacts an election office is willing or required to publish, can an independent party recompute the public count claims and locate failures at a precise layer?

The design is guided by three roles. CANVASS keeps status transitions grounded in election administration: unofficial, canvassed, recounted, amended, and certified totals are not interchangeable. TALLY keeps ballot and contest accounting explicit: batches, reporting units, residual counts, write-ins, and vendor source distinctions must survive normalization. VAULT keeps public verification from becoming a coercion channel: RCOUNT can support inclusion and tamper evidence, but it must not let a voter prove candidate choices.

This paper makes four contributions. First, it defines the RCOUNT package model as a layered public-evidence artifact. Second, it describes the verifier contract currently

implemented in the Rust crates. Third, it identifies the optional boundary with RPLAN district assignments. Fourth, it documents the synthetic positive and negative fixture suite used to stabilize the first contract before real state or vendor adapters are introduced.

2 Count Semantics

RCOUNT v0 is a summary-level substrate. It does not attempt to preserve the full voting-system object model, but it must name the objects it summarizes so that later adapters do not erase election meaning.

Term	Meaning in the RCOUNT boundary
Ballot	A voter-facing voting record as treated by the jurisdiction. RCOUNT v0 does not model voter eligibility or ballot issuance decisions.
Ballot card	A physical or electronic card that may contain only part of a ballot. Future adapters must not assume one CVR row equals one voter.
CVR row	A voting-system export row describing interpreted marks. RCOUNT v0 can cite CVRs as source evidence but normalizes only public summaries.
Contest	An election question with valid selections, vote-for rules, and residual accounting.
Selection	A candidate, option, or write-in bucket that may receive votes in a contest.
Summary	A public count claim: selections plus undervotes, overvotes, blank contests, and counted ballots for one contest, reporting unit, status, and optional batch.
Batch	An accounting control such as election-day, mail, provisional, or central-count ballots, with accepted, counted, and rejected counts.
Reporting unit	The public geography or accounting unit: precinct, split precinct, vote center, batch, jurisdiction total, or district total.

Residual counts are first-class because they change the meaning of the total. For a single-vote contest, the local equation is:

$$\text{counted ballots} = \sum \text{selection votes} + \text{undervotes} + \text{overvotes} + \text{blank contests}.$$

Write-ins may be reported as explicit candidates, adjudicated names, or a bucket depending on jurisdiction and export format. RCOUNT v0 uses a write-in-bucket selection in its synthetic fixtures and leaves richer write-in normalization to adapters.

Batch accounting is a separate control from contest arithmetic. In the `mail-batch-added` fixture, accepted ballots reconcile as:

$$\text{accepted ballots} = \text{counted ballots} + \text{rejected ballots}.$$

A batch summary must then agree with the batch manifest's counted ballots. This distinction lets RCOUNT catch a missing or inconsistent batch without treating a legitimate late mail batch as suspicious merely because a total changed.

3 Package Model

An RCOUNT package is a directory whose files separate source evidence, normalized records, reconciliation declarations, proof sketches, status events, and transcripts. The current Rust implementation writes these layers as JSON or NDJSON files so they can be diffed, hashed, and replayed with ordinary tools.

Layer	Required	Purpose
`manifest.json`	Yes	Jurisdiction, election metadata, version, and package hash
`sources/`	Yes	Raw or synthetic source evidence and package-relative source index hashes
`normalized/contests.ndjson`	Yes	Contest definitions and valid selections
`normalized/reporting-units.ndjson`	Yes	Precincts, batches, jurisdiction totals, districts
`normalized/summaries.ndjson`	Yes	Contest totals by reporting unit, status, and optional batch
`normalized/batches.ndjson`	Optional	Batch manifests and accepted/counted/rejected ballots
`normalized/lineage.ndjson`	Optional	Reporting-unit split, merge, and unchanged events
`proofs/`	Optional	Privacy-safe inclusion proof sketches and package hashes
`status/events.ndjson`	Yes	Canvass, correction, recount, and certification events
`transcripts/`	Yes	Verifier output and replay evidence

The central record is a summary: a contest id, reporting unit id, optional batch id, count status, selection totals, undervotes, overvotes, blank contests, and counted ballots. This is intentionally not a cast-vote-record schema. Summaries are the public claims being verified; source files and future adapters can explain how those summaries were derived from vendor exports, statements of votes, or canvass reports.

Hashes use domain-separated prefixes so that a record hash, source hash, file hash, proof hash, and package hash cannot be silently substituted for one another. A package content hash binds the normalized model. Source hashes bind the raw evidence bytes used to justify that model. This distinction lets a verifier say whether arithmetic failed, source evidence was tampered with, or a manifest no longer matches the normalized records.

4 Status, Schema, and Hash Contract

RCOUNT records count status as data rather than prose. A status says what public claim is being replayed; a status event says why the claim changed.

Status	Typical evidence	What RCOUNT can verify
Unofficial	Election-night export or preliminary report	Local arithmetic and source hash stability for the preliminary claim.
Canvassed	Canvass report, board minutes, corrected public export	Arithmetic, jurisdiction totals, and declared correction events.
Recounted	Recount report or updated statement of votes	Arithmetic and event lineage for the recount snapshot.
Amended	Amended certification or court-ordered correction	Arithmetic and public event history for the amended snapshot.
Certified	Official certification artifact	Mechanical consistency of records marked certified, not the legal validity of certification.

The schema contract is intentionally small. A package declares `rcount_version`; the current crates use a draft version while the format is incubating. Required layers are the manifest, contests, reporting units, summaries, status events, source index, source bytes, package hashes, and transcripts. Optional layers include batch manifests, lineage events, inclusion proof sketches, and district aggregation transcripts. Source index paths are package-relative and must remain under `sources/`.

Hashes are over canonical JSON projections or exact source bytes with domain-separated prefixes. The current prefix families are source, record, file, package, event, and proof. This prevents a hash from one layer from being silently reused as evidence for another. A district aggregation transcript adds the RCOUNT package content hash, the RPLAN plan hash, and, when present, the RCTX context hash. Those hashes bind the district total to both the count package and the assignment used to project reporting units into districts.

5 Verifier Contract

The verifier contract is deliberately layered. A package may fail because local contest arithmetic is wrong, because jurisdiction totals do not equal the sum of their reporting units, because a batch summary has no batch manifest, because a lineage event references a missing precinct, because a proof exposes voter choices, or because source bytes no longer match the source index. These are different failures and should not collapse into a generic "invalid election" message.

Check	Inputs	Failure caught
`contest_selection_sum`	Contest, summary	Votes plus residuals mismatch ballots
`jurisdiction_contest_total`	Summaries	Jurisdiction total disagrees with units
`status_event_declared`	Status events	Missing or malformed public event history
`canvass_correction_event`	Status events, summaries	Changed canvass lacks correction event
`accepted_ballots`	Batch manifest	Accepted ballots mismatch counted plus rejected
`batch_summary_total`	Batch manifest, summary	Batch summary lacks accounting evidence
`lineage_conservation`	Reporting units, lineage	Split or merge references missing units
`proof_privacy_gate`	Inclusion proof sketches	Proof reveals choices or linkable identity
`source_hash_match`	Source index, source bytes	Raw evidence changed after packaging
`district_aggregation_total`	RCOUNT, RPLAN	District totals do not replay from units

The current CLI exposes two verifier paths. `rcount verify` reads a package directory and emits a pass/fail transcript with per-check results. `rcount aggregate-districts` reads an RCOUNT package and an RPLAN assignment, then emits district totals and an aggregation transcript. The latter path is intentionally optional: base RCOUNT packages do not require districts, districting graphs, or RPLAN contexts.

The contract is not a certification standard. A passing transcript means the package satisfied the declared mechanical checks. It does not prove that every eligible voter was registered, that every ballot was accepted or rejected lawfully, that voting-system software was uncompromised, or that a court would uphold the election. Those questions belong to law, administration, security, and audit practice. RCOUNT contributes a replayable evidence layer underneath those decisions.

6 Synthetic Evaluation

The first RCOUNT milestone uses synthetic fixtures instead of real election exports. This is a conscious staging decision. Real data adapters introduce jurisdiction and vendor variation before the core contract is stable. Synthetic fixtures let each verifier layer be exercised with minimal records, known expected outcomes, and reproducible failure modes.

Fixture	Expected	Contract layer
`summary-basic`	pass	Contest sums and jurisdiction total
`canvass-correction`	pass	Status transition and correction event
`mail-batch-added`	pass	Batch accounting and late mail evidence
`precinct-split-lineage`	pass	Split and merge reporting-unit lineage
`privacy-inclusion-sketch`	pass	Choice-free inclusion proof sketch
`district-aggregation-rplan`	pass	Optional RPLAN district aggregation
`multi-election-harness`	pass	Three-cycle L2 replay with lineage and districts
`bad-selection-sum`	fail	Local contest arithmetic
`missing-batch`	fail	Missing batch evidence
`bad-lineage`	fail	Missing current lineage unit
`choice-bearing-proof`	fail	Receipt-risk proof exposure
`tampered-source`	fail	Source hash mismatch
`missing-source-hash`	fail	Omitted source evidence
`multi-election-harness-negatives`	fail	Bad lineage, stale plan, tampered cycle source

Level	Scope	Evidence in this draft
L0	Pure equations and hash projections	Core unit tests for contest sums, jurisdiction totals, status events, package hashes, batch accounting, lineage, and proof privacy.
L1	Package fixtures and CLI verification	Golden packages under `docs/examples/rcount-golden-packages` and `rcount verify`.
L2	Multi-election replay	Three-cycle synthetic harness with split/merge lineage, per-cycle RPLAN plans, district aggregation, and negative cases.

The L2 harness is the most important fixture because it links the concepts that otherwise look separate. It generates three synthetic election cycles. The 2026 cycle splits one precinct into two. The 2028 cycle merges two precincts into a new reporting unit. Each cycle has a package, a plan assignment, and district aggregation evidence. Negative variants then break the lineage, use a stale RPLAN unit, or tamper with one cycle's source bytes.

This style of synthetic evaluation is not a claim about performance or real-world coverage. It is a contract test. Before RCOUNT reads a public state file, the package format should already know how to fail when arithmetic, lineage, privacy, district projection, or source evidence breaks.

The reproducibility surface is executable. The key commands are:

```
cargo run -p rcount-district --example multi_election_harness
cargo run -p rcount-district --example multi_election_negative_harnesses
cargo test -p rcount-district -p rcount-audit -p rcount-cli
```

Negative fixtures intentionally fail through different surfaces:

- bad lineage fails the package verifier;

- stale plan units fail district aggregation;
- tampered cycle sources fail source hash verification.

7 Boundaries and Non-Goals

RCOUNT is most useful when its boundaries are explicit. It is not an election management system, a voter registration database, a ballot marking device verifier, an end-to-end voting protocol, or a replacement for risk-limiting audits. It can store evidence relevant to those domains, but its core claim is that public count packages can be replayed and checked.

The certification boundary is especially important. A canvassing board may certify results after considering statutes, deadlines, cure rules, recount requests, court orders, and evidence that cannot be reduced to arithmetic. RCOUNT can show that a canvassed total equals a set of public summaries and that a correction event exists. It cannot decide whether the board acted lawfully.

The privacy boundary is equally strict. A public artifact that lets a voter prove their candidate choice can become a coercion receipt. RCOUNT therefore treats inclusion proofs as privacy-gated sketches: they may show that an anonymized token exists in accepted or counted evidence, but they must not carry candidate selections or link voter identity, ballot style, and timestamp in a way that reidentifies the ballot.

Threat or claim	RCOUNT v0 position	Non-goal or warning
Source tampering	Source bytes are hashed and checked against the source index.	Hashes do not prove the original source was truthful.
Parser substitution	Normalized records are package-hashed.	Parser diagnostics and independent parser disagreement are future adapter work.
Package truncation	Package content hash changes when normalized records are omitted.	Detached signatures are future work.
Equivocation	A transcript binds one package hash to one verification run.	Public log consistency is not yet implemented.
Small-cell disclosure	Proof sketches reject obvious linkable identity fields.	Rare write-ins, tiny precincts, ballot styles, and timestamps still require suppression policy.
Coercion receipts	Choice-bearing proof sketches fail the privacy gate.	RCOUNT must not let voters prove candidate choices.
Malware resistance	Out of scope.	RCOUNT is not a voting-system security proof.

The RPLAN boundary is optional. District vote aggregation is necessary when precinct or reporting-unit totals must be projected onto districts, especially when district assignments change across cycles. But count packages should not depend on districting machinery unless the public claim being verified is a district total. This keeps the base verifier small while allowing RCOUNT and RPLAN to meet at a hashed assignment boundary.

RCOUNT can mechanically show	RCOUNT does not show
Package records match their manifest hash.	Election officials acted lawfully.
Source bytes match the declared source index.	Voting-system software was uncompromised.
Contest and jurisdiction arithmetic reconciles.	Every eligible voter was registered or every ballot decision was correct.
Declared batches, lineage, and privacy gates pass.	Public proof sketches are a complete end-to-end voting protocol.
District totals replay from a specified RPLAN assignment.	The district plan is fair, lawful, or politically neutral.

8 Conclusion

RCOUNT proposes a modest but useful fixed point for public election-count evidence: package the normalized claims, bind them to source bytes, replay the arithmetic, preserve canvass and precinct lineage, reject unsafe proofs, and emit transcripts that name the layer that passed or failed. The current Rust implementation and fixture suite are intentionally synthetic, but they cover the first contract surface from local summaries through multi-election district aggregation.

The next papers in the V-series can now specialize. V.1 can focus on canvass arithmetic and status transitions. V.2 can focus on precinct lineage across cycles. V.3 and V.4 can separate tamper evidence from privacy-safe public inclusion. V.8 can give the RPLAN district aggregation boundary its own full treatment. The shared rule across the series is simple: make public arithmetic replayable without overstating what software can certify.

References

Gio Della-Libera. Rcount substrate specification, 2026a. Working specification in docs/specs/2026-05-12-rcount-substrate.md.

Gio Della-Libera. Rplan and rctx reproducible district plan artifacts, 2026b. Project documentation and Rust crates.